01 MODULES IN NESTJS — THEORY

A Module is a class that groups related controllers, providers (services), and other pieces into one cohesive unit. Every NestJS app has at least one module — the root AppModule — and larger apps split into one module per feature (UserModule, EmployeeModule, OrderModule, etc.).

Think of a module as a box that says: "these controllers, these services, and these imported modules belong together as one feature."

```
AppModule (root)
  -> imports EmployeeModule -> (contains EmployeeController, EmployeeService)
  -> imports UserModule     -> (contains UserController, UserService)
```

1. DECLARING A MODULE

```
import { Module } from '@nestjs/common';

@Module({})
export class EmployeeModule {}
```

This is exactly what src/employee/employee.module.ts looks like right now in this project — a valid but EMPTY module. It compiles and can be imported, but it doesn't do anything yet because none of the @Module() metadata keys have been filled in.

2. THE FOUR METADATA KEYS

```
@Module({
  imports:     [OtherModule],          // other modules this one needs
  controllers: [EmployeeController],    // handles incoming HTTP requests
  providers:   [EmployeeService],       // services/DI-managed classes
  exports:     [EmployeeService],       // what other modules may use
})
export class EmployeeModule {}
```

Key	Meaning
imports	Other modules whose EXPORTED providers this module wants to use (e.g. a shared DatabaseModule).
controllers	Route classes (@Controller) that belong to this feature.
providers	Services (@Injectable) that belong to this feature; Nest instantiates these and injects them where needed.
exports	Subset of "providers" that OTHER modules are allowed to inject after importing this module. Anything not listed here is private to this module.

3. FILLING IN THIS PROJECT'S EmployeeModule

Right now EmployeeModule is a stub. Once you add EmployeeController and EmployeeService (e.g. via "nest g controller employee" / "nest g service employee"), it should look like this:

```
import { Module } from '@nestjs/common';
import { EmployeeController } from './employee.controller';
import { EmployeeService } from './employee.service';

@Module({
  controllers: [EmployeeController],
  providers: [EmployeeService],
})
export class EmployeeModule {}
```

No "imports" or "exports" needed yet — this module is self-contained.

4. HOW A MODULE GETS UTILIZED (WIRED INTO THE APP)

A module does nothing by sitting alone — it must be imported by another module, ultimately reaching the root AppModule, which is what Nest bootstraps in main.ts. This project already does this:

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UserController } from './user/user.controller';
import { UserService } from './user/user.service';
import { EmployeeModule } from './employee/employee.module';

@Module({
  imports: [EmployeeModule],
  controllers: [AppController, UserController],
  providers: [AppService, UserService],
})
export class AppModule {}
```

- EmployeeModule is listed in AppModule's "imports" array.
- Once EmployeeController/EmployeeService are added to EmployeeModule (as in section 3), Nest will automatically register EmployeeModule's routes and make its providers available — all through this one import.
- Notice UserController/UserService are registered directly on AppModule instead of having their own UserModule. That works for small apps, but as a feature grows, pulling it into its own module (like EmployeeModule) keeps AppModule from becoming a dumping ground.

5. EXPORTING A PROVIDER FOR USE IN ANOTHER MODULE

If, say, OrderModule needed to look up employee data, EmployeeService must be exported first:

```
// employee.module.ts
@Module({
  controllers: [EmployeeController],
```

```
  providers: [EmployeeService],
  exports: [EmployeeService],        // <-- makes it available elsewhere
})
export class EmployeeModule {}

// order.module.ts
@Module({
  imports: [EmployeeModule],          // pulls in its exported providers
  controllers: [OrderController],
  providers: [OrderService],
})
export class OrderModule {}

// order.service.ts
@Injectable()
export class OrderService {
  constructor(private readonly employeeService: EmployeeService) {}
  // Nest can inject EmployeeService here ONLY because EmployeeModule
  // exported it AND OrderModule imported EmployeeModule.
}
```

Without "exports", EmployeeService stays private — other modules can import EmployeeModule all they want, but Nest will refuse to inject EmployeeService anywhere outside EmployeeModule itself.

6. SHARED / GLOBAL MODULES (COMMON PATTERN)

Things like a database connection or config service are needed almost everywhere. Two common approaches:

- Export it from a module and have every feature module import that module explicitly (most common, explicit, easy to trace).
- Mark a module @Global() so any module can inject its exports WITHOUT importing it — use sparingly, it hides where a dependency comes from:

```
import { Module, Global } from '@nestjs/common';

@Global()
@Module({
  providers: [ConfigService],
  exports: [ConfigService],
})
export class ConfigModule {}
```

7. NEST CLI — GENERATING A MODULE

```
nest generate module <name>
nest g mo <name>              (short form)
```

Example:

```
nest g mo employee
```

This creates src/employee/employee.module.ts with an empty @Module({}) shell (exactly the file already in this project) and, if AppModule is in the default location, automatically adds it to AppModule's "imports" array.

```
nest g resource employee      Generates module + controller + service
                               + DTOs in one shot, fully wired together
                               — the fastest way to start a new
                               feature from scratch.
```

★ RULE OF THUMB

One feature = one module = one folder (controller + service + DTOs + module, all under src/<feature>/). The module file is the "table of contents" for that folder: it says what routes exist (controllers), what logic exists (providers), what it borrows from elsewhere (imports), and what it's willing to share (exports).